SPECIAL PROJECT · DRAFT V0.1 · 5 NOVEMBER 2026

Agent OS: specification (GrapheneOS fork route)

Draft v0.1, 5 November 2026. Owner: Mohammed Husari. Companion deck: `graphene_agent_os.pdf`.

---

1. Summary

A rebranded fork of GrapheneOS for Pixel phones, signed with our own keys, with an AI agent built into the operating system as a privileged component. The agent can see every screen and act in every app, is invoked by voice or text, and runs its models on the device, on a GPU server we own, or (opt-in, off by default) in the cloud. Policy and confirmation are enforced below the agent, inside `system_server`, so neither the agent nor content it reads can bypass them.

Design rule that everything else follows from: **the agent can touch anything on the phone except the confirmation dialog.**

---

2. Goals and non-goals

Goals

1. Prompt the phone in natural language, by voice (including a wake word with the screen off) or text, and have it carry out multi-step tasks.
2. Drive **any** installed app without per-app integration: read its UI, tap, swipe, type, navigate.
3. Full control of the OS and device. No third party decides what the agent may see or do.
4. Local-first: models on the device or on user-owned hardware. Nothing leaves the phone unless the user has allowed it, per app and per role.
5. Safe against hostile content: text in a message, web page or screen can never trigger an action by itself.
6. Keep GrapheneOS's hardening and update cadence.

Non-goals

- Faking a different device or OS to pass attestation, Play Integrity or app security checks.
- Automating logins, payments, biometric or identity verification without a real touch.
- Hiding the agent from apps' anti-fraud detection.
- Replacing Android apps with a Linux environment.
- Upstreaming the agent into GrapheneOS (narrow features may be proposed later; see §12).

3. Users and scope

v1 target: a single owner's personal device(s). Distribution to other users is a later decision (§12) and changes the risk profile (Doubao precedent, §11).

---

4. Hardware

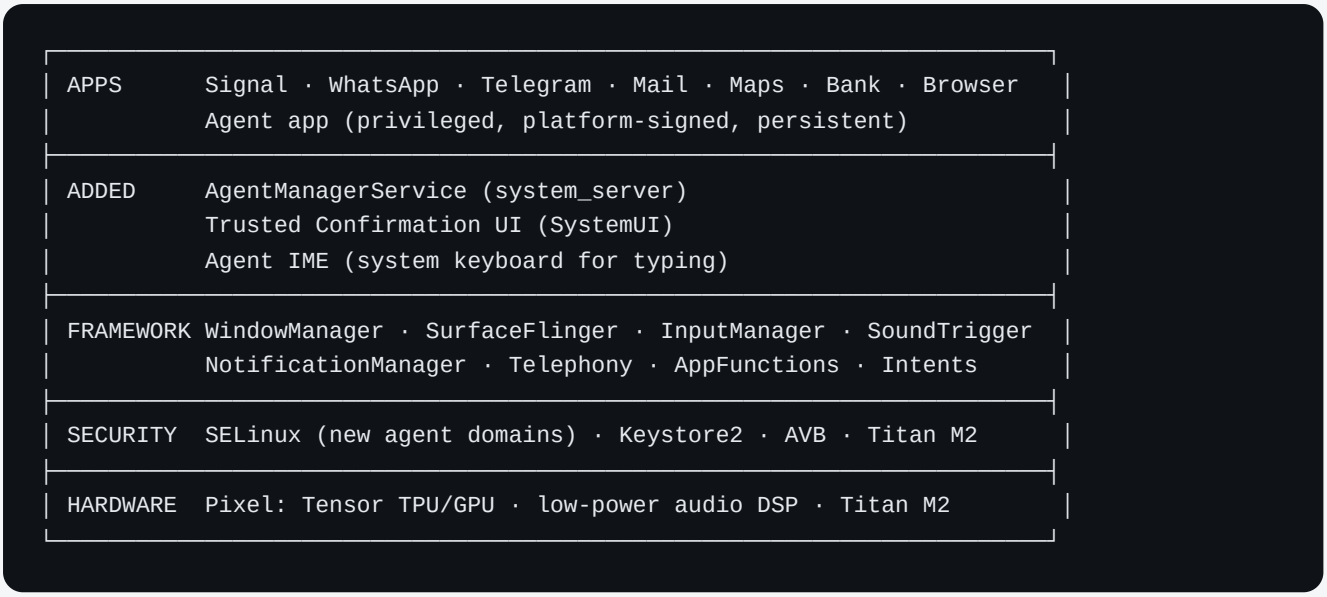
ITEM	CHOICE	NOTES
Phone	Pixel 9 Pro or Pixel 10 Pro, 16 GB RAM	**Open.** 16 GB needed for a 7–8B planner on-device. GrapheneOS supports Pixel 6–10a.
Dev devices	2–3 Pixels of the same model	One daily driver, one for CI/tests, one spare for bricked-build recovery
Build machine	64 GB+ RAM, 16+ cores, ~500 GB NVMe	Cloud instance acceptable; AOSP full build ~1–2 h
Home inference server	GPU with 24 GB VRAM minimum, 32 GB+ preferred	**Open.** Runs vision + planner models; reached over WireGuard
Key storage	Offline (YubiKey/HSM or air-gapped machine)	Platform, verified-boot (AVB) and OTA signing keys

---

5. Base OS

- Fork of GrapheneOS at its current stable tag, tracking stable releases.
- Rebranded (name, boot animation, update server URL, package names for added components). GrapheneOS's name and trademarks are not used.
- Everything GrapheneOS ships is kept unchanged except the additions in §6–§9. Sandboxed Google Play remains available per profile.
- Our own AVB key, platform key, and OTA signing key; bootloader relocked with our AVB key.
- Our own OTA update server (same mechanism GrapheneOS uses: `Updater` app pointing at our URL).

6. Architecture



Three components are added:

6.1 AgentManagerService (`com.<brand>.agent` in `system_server`)

The single gateway between the agent and every privileged capability. The agent app has **no** direct dangerous permissions; it holds only a signature permission to bind to this service. All of the following go through it:

- Screen observation: UI tree (view hierarchy) and screenshot capture, including FLAG_SECURE surfaces.
- Input injection: taps, long-presses, swipes, key events, text via the Agent IME.
- Notifications: read, dismiss, inline reply.
- Telephony and SMS: place calls, send SMS (confirmation-gated).
- App launch, intents, deep links, AppFunctions calls.
- App-data read (per-app grant only, see §9.3).
- Model routing decisions (§8).
- Policy store, action log, trusted confirmation.

Exposed to the agent app via AIDL (`IAgentManager`): `observe()` , `act(Action)` , `requestConfirmation(ActionSummary)` , `route(Role, Payload)` , `getPolicy()` , plus a callback for policy changes and task interruption.

6.2 Trusted Confirmation UI (SystemUI)

A full-screen dialog drawn by SystemUI on request of AgentManagerService. Properties:

- Shows a plain-language summary of the pending action (what, in which app, to whom, amount if any).
- **Rejects injected input.** Only events from a physical touch source (`InputDevice.SOURCE_TOUCHSCREEN` with a real device id, not `INJECT_EVENTS`) count. Implemented in InputDispatcher: while the dialog is up, injected events are dropped.

- Cannot be overlaid (`FLAG_SECURE` , `HIDE_OVERLAY_WINDOWS`), cannot be dismissed by the agent, times out to "deny".
- Required for: send message, send email, call, SMS, payment or checkout, delete, post publicly, install or uninstall, change settings, grant policy, anything the policy store marks as confirm-required.

6.3 Agent app (privileged system app)

- Platform-signed, in `/system_ext/priv-app` , whitelisted in `privapp-permissions` , `persistent="true"` .
- Contains: voice pipeline (wake word hand-off, STT, TTS), planner loop, reader, skills library, UI (chat/voice overlay, policy screen, action log viewer), model backends.
- Holds the default assistant role (`VoiceInteractionService`) for long-press invocation.

---

7. Capabilities (what the agent can see and do)

7.1 Observation

SOURCE	MECHANISM	USE
UI tree	`WindowManagerService` → `AccessibilityNodeInfo` fetched via system path (no third-party accessibility service)	Primary for standard Android UIs; cheap; sent as text
Screenshot	`SurfaceFlinger` capture with `CAPTURE_SECURE_VIDEO_OUTPUT` ; downscaled to ≤1280px long edge	Fallback for Flutter, React Native, WebView, games; any vision task
Notifications	System-level `NotificationListenerService` equivalent inside AgentManagerService	Inbox, bank alerts, inline reply
App-data	Direct file reads in a dedicated SELinux domain, per-app grant	Telegram (TDLib embedded instead), others case by case

Flutter apps only populate the semantics tree when the system reports accessibility enabled; AgentManagerService can set this **per app** (via `AccessibilityManager` flags scoped to a package) rather than globally, since some banks detect the global setting.

7.2 Action

ACTION	MECHANISM
Tap, long-press, swipe, drag	`InputManager.injectInputEvent` with `INJECT_EVENTS`; human-like timing and jitter (§11)
Type	Agent IME (system `InputMethodService`, platform-signed) committing text; falls back to key events
Navigate	Back/Home/Recents key events; launch via `ActivityManager`; deep links; `AppFunctionManager`
Notification reply	`RemoteInput` on the notification's reply action
Calls, SMS	TelecomManager / SmsManager, confirmation-gated
Profiles	Agent may act across user profiles if the policy allows (default: current profile only)

7.3 Voice

- **Wake word:** `SoundTrigger` HAL with `CAPTURE_AUDIO_HOTWORD`, running on the low-power audio DSP; works with the screen off. Model: openWakeWord or a custom keyword model compiled for the DSP. **On-device only, not routable.**
- **STT:** whisper.cpp (small/medium) on-device. **On-device only, not routable.**
- **TTS:** Piper or RHVoice on-device; eSpeak NG fallback.
- Push-to-talk via long-press home gesture (assistant role) and a Quick Settings tile.

7.4 Skills library

- Each successfully completed task is saved as a skill: ordered steps with, for each, the expected screen signature (package, activity, key UI nodes) and the action taken.
- Replay checks the signature before each step; on mismatch the planner takes over from that step and the skill is updated.
- Skills are per app and per app version; stored encrypted in the agent's data dir; exportable.
- Replay needs no model calls.

---

8. Model routing

8.1 Backends

BACKEND	RUNTIME	DEFAULT
On-device	LEAP (LFM2/LFM2.5) or llama.cpp with Tensor TPU/GPU delegate; agent added to GrapheneOS's TPU allowlist	Always present
Home server	vLLM or llama.cpp server behind an OpenAI-compatible API, reached over WireGuard with a device-bound key; no logging, no persistence on the server	Optional
Cloud	Any OpenAI-compatible endpoint	Optional, **off by default**

8.2 Roles

ROLE	WHAT IT DOES	DEFAULT ROUTE	USER-CHANGEABLE
Wake word	Keyword spotting	Device	No
STT	Speech → text	Device	No
Reader	Summarises or drafts from untrusted content; **no tool access**	Device	Yes
Vision	Screen → structured description / grounding	Home → device fallback	Yes
Planner	Decides the next action; the only role that can call tools; sees labels and structured observations, not raw untrusted text	Home → device fallback	Yes

Candidate models to evaluate: vision/grounding: UI-TARS, Qwen-VL, AgentCPM-GUI; planner: LFM2.5-8B-A1B on device, a 30–70B instruct model on the home server; reader: LFM2.5 small on device.

8.3 Routing rules (enforced in AgentManagerService)

1. **Per-app rules win.** e.g. "banking category and Signal: all roles on-device". Observations from such apps are never serialised to a remote backend.
2. **Per-connection rules.** Home server allowed on: home Wi-Fi (SSID/BSSID pinned), optionally mobile data above a throughput threshold.
3. **Fallback.** Remote unreachable or over latency budget (default 3 s) → on-device model, task continues.
4. **Indicator.** Status-bar icon showing the current backend (phone / home / cloud), implemented like the mic/camera privacy indicators; cannot be hidden by the agent.
5. **Payload minimisation.** Prefer UI tree text over screenshots; downscale screenshots; strip notification bodies from apps marked device-only.

9. Policy and safety model

9.1 Per-app grants

One policy store in AgentManagerService, one screen in Settings. Per app (and per profile):

- Observe: none / notifications only / UI tree / UI tree + screenshot
- Act: none / navigate only / full input
- App-data read: off / on
- Model route: default / device-only
- Confirmation: default list / always confirm / never automate

Defaults for a fresh install: messaging apps = observe + act, confirm sends; banking category = notifications only, device-only, never automate; everything else = observe + act with the default confirmation list.

9.2 Planner / reader split

Untrusted content (message bodies, web pages, screen text from other apps) is only ever fed to the reader role. The planner receives labels and structured observations ("3 unread from Ali in Signal", "button: Send at (x,y)") and the reader's outputs tagged as untrusted data. The planner is the only role with tool access. This is enforced by AgentManagerService constructing the prompts, not by the agent app.

9.3 Direct app-data access

Off by default. Granting it for an app creates an SELinux allow rule for the agent domain on that app's data label at runtime (via a policy module loaded by AgentManagerService). Signal's database remains unreadable without its Keystore key; we do not modify `keystore2`. Telegram is accessed through TDLib instead.

9.4 Action log

Every observation request, action and confirmation decision is appended to a local, encrypted, append-only log with timestamps and the originating task. Viewable in Settings; exportable.

9.5 Biometrics and secure hardware

Fingerprint/face prompts, Titan-backed Keystore operations and DRM paths are untouched. The agent can bring up a biometric prompt but can never satisfy it.

---

10. Build, signing and updates

1. Clone GrapheneOS at the chosen stable tag; apply our patch series (kept as a rebase-able branch, one commit per feature).

2. Generate keys offline: AVB, platform, releasekey, OTA. Store offline. Signing happens on an isolated signer, not the build host.
3. Build `release` variant; sign with `script/release.sh` equivalent; produce factory image + OTA package.
4. OTA server: static host serving the same JSON/zip format the GrapheneOS Updater expects; Updater pointed at our URL.
5. CI: on every GrapheneOS stable tag, auto-rebase the patch series, build, flash the CI Pixel, run the capability suite (§13), and only then publish the OTA.
6. Rollback: keep the previous OTA; A/B slots give automatic fallback on boot failure.

---

11. Risks and mitigations

RISK	MITIGATION
Prompt injection via screen/message content	Planner/reader split; trusted confirmation ignores injected input; confirmation required for all irreversible actions
Server-side bot detection (Doubao precedent)	Human-like input timing (randomised 80–400 ms taps, scroll inertia, no perfectly regular cadence); no automation of logins/payments/identity checks; stop automating any app that objects; never spoof device identity
Bank attestation fails on our key	Accept. Apps that check nothing work; apps needing Play Integrity already fail on GrapheneOS; only banks that specifically whitelist GrapheneOS's key are lost. Check the user's banks before Phase 1
Update burden	CI auto-rebase + capability suite as gate; one commit per feature to keep conflicts local
Model capability on long unfamiliar tasks	Skills library; home-server planner; task decomposition with per-step verification
Home server compromise	WireGuard with device-bound key; no persistence on the server; per-app device-only rules for sensitive apps
Agent app compromise	It holds no dangerous permissions itself; everything goes through AgentManagerService policy; SELinux-confined
Bricking devices	Spare device; A/B slots; factory images retained

---

12. Open decisions

1. **Audience:** personal only vs product vs GrapheneOS-team collaboration. Affects licence, polish, distribution and the bot-detection risk.
2. **Pixel model** (9 Pro vs 10 Pro) and **home GPU** (24 vs 32+ GB VRAM).
3. **GrapheneOS team:** ask about open-source STT/TTS timeline, planned accessibility safeguards, Android 18 DMA API handling, and fork advice on hardening and cadence. Possible

narrow upstream proposal later: a user-granted, per-app "assistant" permission with a visible indicator.

4. **Which apps matter most:** daily jobs; banking/fintech apps in use (check each against §11 attestation row); what the "FT app" is.
5. **Brand name** for the fork.

---

13. Phases and acceptance

PHASE	DELIVERABLE	ACCEPTANCE TEST	TIME
1. Own the pipeline	Stock GrapheneOS built, signed with our keys, flashed, bootloader relocked, OTA from our server	Phone boots locked with our key; an OTA from our server installs and boots	Weeks 1–2
2. Agent v0	AgentManagerService, agent app, UI tree + screenshot, INJECT_EVENTS, Agent IME, notifications, on-device model	By voice: read unread Signal/WhatsApp/Telegram, draft and (after confirmation) send a reply in each	Weeks 3–8
3. Safety in the OS	Trusted Confirmation UI, policy store + Settings screen, planner/reader split, action log; home-server backend with fallback and indicator	Injection test: a message instructing "send €100 to X" produces no action and no confirmation prompt; confirmation dialog ignores injected taps; routing rules observed in logs	Weeks 9–12
4. Any app	Wake word on DSP, telephony/SMS, skills library, per-app Flutter semantics toggle, cross-profile option	Capability suite: set tasks in ~20 apps across messaging, email, maps, shopping, delivery, banking (read-only), browser, Flutter app, React Native app, game; ≥80% pass; skill replay with 0 model calls	Months 4–5
5. Sustain	CI auto-rebase on each GrapheneOS stable tag gated by the suite; distribution decision; possible DMA-API app route for stock Android (2027)	Two consecutive upstream releases shipped within 7 days with the suite green	Ongoing

---

14. Team and cost

- 1–2 engineers with Android platform experience (system services, AIDL, SELinux, Soong, SystemUI). One more for the model/agent side.
- Build box (cloud): ~$200–400/month. Devices: 2–3 Pixels. GPU server: existing or ~$2–4k.
- Ongoing: roughly 1–2 engineer-days per GrapheneOS release once CI is in place.

15. Sources

- GrapheneOS: features, build and signing docs, usage guide (accessibility warning), FAQ; TPU allowlist toggle.
- AOSP: `INJECT_EVENTS`, `CAPTURE_SECURE_VIDEO_OUTPUT`, `CAPTURE_AUDIO_HOTWORD`, SoundTrigger HAL, `AppFunctionManager`, privapp-permissions, AVB.
- Gemini screen automation (Google support, Feb 2026); Doubao / Nubia M153 coverage (Yicai, Caixin, 36Kr, Dec 2025); EU DMA decision on Android AI access (16 July 2026).
- Liquid LEAP + LFM2.5; whisper.cpp; openWakeWord; Piper; UI-TARS; Qwen-VL; AgentCPM-GUI; TDLib.

Agent OS specification, draft v0.1. Companion deck: graphene_agent_os.pdf. Draft for discussion; please don't forward further without asking.